sim6809

A headless Motorola 6809 simulator – user manual

plm09 project

October 2026

Contents

1 Introduction **2**

2 Building **2**

3 Command-line usage **3**

3.1 Example 4

4 Loading programs **4**

5 The machine **5**

5.1 Memory 5

5.2 Initial state 5

5.3 Memory-mapped I/O 5

6 The CPU **6**

6.1 Instruction set 6

6.2 Interrupts 6

6.3 Instructions that stop the simulator 7

7 OS-9 mode **7**

8 FLEX mode **10**

9 Board devices **12**

10 Exit status **13**

11 Tracing **13**

12 Using sim6809 with plmc **14**

13 Verification **14**

14 Limitations **15**

15 Source code **15**

1 Introduction

`sim6809` is a command-line simulator for the Motorola MC6809 8-bit microprocessor. It loads a program in Motorola S-record format, runs it with no display or user interface, and reports the result through standard output and its exit status. This makes it a good fit for automated tests, scripts and continuous integration.

It was written for the plm09 project, a compiler that turns a subset of PL/M into 6809 assembly for the lwasm assembler. In that project it runs every compiled test program. Nothing in it depends on the compiler, though: it runs any 6809 program that can be expressed as S-records and does its I/O through the simulator's I/O block.

Main features

- The complete documented MC6809 instruction set, including all addressing modes (indexed, indirect and PC-relative) and the flag behavior given in the Motorola datasheet.
- A 16-byte memory-mapped I/O block for character output and input, printing a 16-bit decimal number, and ending the program with an exit status.
- An optional per-instruction trace with disassembly and register contents.
- An instruction limit, so a runaway program can't hang a test run.
- Distinct exit statuses for normal exit, illegal instruction and timeout.
- Memory filled with `$A5` at startup, so reads of uninitialized variables are easy to spot.
- An OS-9 mode that loads an OS-9 program module at any address, sets up a process the way OS-9's `F$Fork` does, emulates the basic I/O system calls on the terminal and on host files, and enforces memory protection. It's used to check that `plmc --os9` modules really are position-independent.
- Board devices (an MC6850 ACIA, an FT245RL USB FIFO, and working software interrupts) for running small boards' monitors and programs.
- A FLEX mode that loads a FLEX binary (`.CMD`) file, emulates the FLEX DOS entry points programs call, and protects FLEX's memory.

What it deliberately leaves out is covered in Limitations.

2 Building

`sim6809` is plain C11 with no dependencies beyond the C library. From the top of the plm09 tree:

```
make build/sim6809
```

`make install` copies it, with plmc and srec2flex, to `BINDIR` (default `/usr/local/bin`; for example `make install BINDIR=~/bin`), and the man page `sim6809(1)`, a summary of the options, to `MANDIR/man1` (default `/usr/local/share/man`). `make rpm` builds a Fedora RPM of all of plm09.

`make sim-test` builds it and runs its self-tests (see Verification). Assembling the test programs needs lwasm from lwtools, and the instruction length test also needs python3.

3 Command-line usage

```
sim6809 [options] prog.s19
```

Option	Meaning
`-t`	Trace every instruction to standard error before executing it. See Tracing.
`-n N`	Stop after *N* instructions. The default is 100,000,000.
`--io BASE`	Put the I/O block at *BASE* instead of `$FF00`. *BASE* can be at most `$FFF0`.
`--entry ADDR`	Start execution at *ADDR*. Needed only if the file has no S9 record; if given, it overrides the S9 address.
`--zero`	Clear memory to `$00` instead of filling it with `$A5`.
`--os9`	The file is an OS-9 program module, not S-records. See OS-9 mode.
`--load ADDR`	With `--os9`: load the module at *ADDR* (default `$2000`). Any address works.
`--data ADDR`	With `--os9`: put the data area at *ADDR*, which must be a multiple of 256 (default `$8000`).
`--param STR`	With `--os9`: the parameter string passed to the program (default empty).
`--flex`	The file is a FLEX binary (`.CMD`) file, not S-records. See FLEX mode.
`--swi`	Execute `SWI`, `SWI2` and `SWI3` as the 6809 does, through their vectors, instead of stopping. See Board devices.
`--acia ADDR`	An MC6850 ACIA at *ADDR* (status/control) and *ADDR*+1 (data), on stdin and stdout. See Board devices.
`--ft245 ADDR`	An FT245RL USB FIFO at *ADDR*, on stdin and stdout, driving FIRQ. See Board devices.
`--stop-at-end`	With `--acia`: a program waiting for input after the end of stdin ends the run (status 0). See Board devices.
`--reset`	Start at the address in the reset vector (`$FFFE`), as the 6809 does, instead of an S9 record; for ROM images.
`--rom ADDR`	Memory from *ADDR* to `$FFFF` is ROM: writes to it are ignored (except to the I/O block).
`-h`, `--help`	Print a usage summary and exit with status 0.
`--version`	Print `sim6809 (plm09)` and the version, and exit with status 0.

Numbers given to options are **decimal** unless they start with `0x` or `$`. For example, `--entry 4096`, `--entry 0x1000` and `--entry '$1000'` all mean the same address, but `--entry 1000` means decimal 1000. Quote `$` values so the shell doesn't expand them.

Exactly one program file is accepted. Anything that isn't a recognized option is taken as the file name. A second file name, an unknown option or a bad number prints the usage summary and exits with status 2.

3.1 Example

```
; hello.asm -- print a string and a number, then exit with status 3
IO      EQU     $FF00
        ORG     $1000
START   LDS     #$8000
        LDX     #MSG
LOOP    LDA     ,X+
        BEQ     DONE
        STA     IO              ; character output
        BRA     LOOP
DONE    LDD     #1234
        STD     IO+4            ; decimal output
        LDB     #3
        STB     IO+2            ; exit with status 3
MSG     FCC     "HELLO"
        FCB     10,0
        END     START
```

```
$ lwasm -9 -f srec -o hello.s19 hello.asm
$ sim6809 hello.s19
HELLO
1234
$ echo $?
3
```

`lwasm -9` keeps lwasm in 6809 mode (by default it accepts 6309 instructions), and `END START` makes lwasm write the S9 start-address record that `sim6809` uses as its entry point.

4 Loading programs

`sim6809` reads Motorola S-records:

Record	Handling
`S0` (header)	ignored
`S1` (data, 16-bit address)	bytes stored in memory at the given address
`S5` (record count)	ignored
`S9` (end, 16-bit address)	sets the start address
`S2`, `S3`, `S7`, `S8` (24/32-bit)	error

Every record's checksum is verified. A malformed line, a bad hex digit or a checksum mismatch stops loading with a message giving the line number, and the simulator exits with status 2. Blank lines and both LF and CR LF line endings are accepted. Data wraps around at `$FFFF`.

If a file contains more than one S9 record, the last one wins. That makes it easy to combine programs: `cat prog.s19 monitor.s19 > both.s19` loads both, and execution starts at `monitor.s19`'s entry point. The plm09 test suite uses this to load a stand-in Color BASIC ROM together with a compiled program.

Intel HEX. Lines starting with `:` are read as Intel HEX records instead, as ROM images are often distributed: type `00` (data) and `01` (end of file), with their checksums verified; other types are an error. They give no start address, so use `--entry` or `--reset`. A file can mix both formats, for example a program's S-records followed by a ROM's HEX file.

lwasm's `RMB` reserves memory without writing any bytes, so uninitialized variables don't make the S-record file larger. They start with the fill value (`$A5`, or `$00` with `--zero`).

5 The machine

5.1 Memory

The 64 KB address space is all RAM, except for the 16-byte I/O block. There's no ROM and no memory protection: programs can write anywhere, including over their own code and the interrupt vectors at `$FFF0`–`$FFFF`. The simulator starts at the entry address, not through the reset vector; the IRQ, FIRQ and NMI vectors are read when an interrupt is taken (see Interrupts).

5.2 Initial state

Register	Value
PC	the S9 address, or `--entry`
A, B, X, Y, U, DP	0
S	**0**
CC	`$50` (I and F set, as after a hardware reset)

S starts at zero, so **a program must load S before pushing anything.** Otherwise the first push wraps around to `$FFFF`/`$FFFE`. (That is ordinary RAM in the simulator, so it doesn't crash, but it isn't what you want.)

5.3 Memory-mapped I/O

The I/O block is 16 bytes long, at `$FF00` by default (see `--io`). Offsets are relative to its base:

Offset	Access	Function
+0	write	Write the byte to standard output.
+1	read	Read one byte from standard input. Returns `$00` at end of input.
+2	write	Stop, with the byte as the simulator's exit status.
+4	write	Store the high byte for +5.
+5	write	Print (high byte from +4) × 256 + this byte as an unsigned decimal number, followed by a newline.
+6	write	Assert interrupt lines: bit 0 IRQ, bit 1 FIRQ, bit 2 NMI.
+6	read	The asserted lines (bit 0 IRQ, bit 1 FIRQ).
+7	write	Release the lines whose bits are set.
+8	write	Store the high byte of the timer period for +9.
+9	write	Start the timer: every (high byte from +8) × 256 + this byte instructions it asserts its lines. 0 stops it.
+10	write	The lines the timer asserts (bits as for +6; default IRQ).

Other offsets, and reads of the write-only offsets, return `$00`. Writes to unused offsets are ignored.

- **Printing numbers:** `STD $FF04` writes A to +4 and then B to +5, so it prints D in one instruction. A single write to +5 prints using whatever +4 last held.

- **Exiting:** the exit write takes effect when the current instruction finishes. Only the first one counts. Standard output is flushed before the simulator exits.
- **Input:** stdin is read through the C library, so input can come from a file or a pipe (`sim6809 prog.s19 < input.txt`). With no input, every read returns 0, just as at end of input.
- **Instruction fetches** always read memory directly and never touch the I/O block, so code that runs into the I/O addresses never consumes input. Every other access goes through the I/O block, including stack pushes and pulls, the bytes of 16-bit operands, and pointers read for indirect addressing.

6 The CPU

6.1 Instruction set

The simulator implements all documented MC6809 instructions, on all three opcode pages (`$10` and `$11` prefixes), with every addressing mode: inherent, immediate, direct (using DP), extended, relative (8- and 16-bit), and indexed.

The indexed forms are:

- constant offsets of 5, 8 and 16 bits from X, Y, U or S;
- accumulator offsets `A,R`, `B,R`, `D,R` (sign-extended);
- auto-increment and auto-decrement `,R+`, `,R++`, `,-R`, `,--R`;
- PC-relative with 8- and 16-bit offsets;
- indirect versions of every form that has one, plus extended indirect `[addr]`.

The condition codes follow the Motorola datasheet. Where the datasheet leaves a result undefined, the simulator behaves as follows:

Case	Behavior
H after subtract and compare	unchanged
V after `DAA`	cleared
V after `SEX`	unchanged
`TFR`/`EXG` from an 8-bit to a 16-bit register	high byte is `$FF` (as on the real MC6809)
`TFR`/`EXG` from a 16-bit to an 8-bit register	low byte is used
`TFR`/`EXG` naming an invalid register code	reads `$FF`/`$FFFF`, writes are ignored

`RTI` honors the E flag: it pulls the whole register set when E is set, and only CC and PC when it is clear.

6.2 Interrupts

Programs raise interrupts themselves, through the I/O block (offsets +6 to +10), standing in for devices:

- **IRQ and FIRQ are level-triggered**, like the 6809's pins: a line stays asserted until the program releases it (+7), which plays the part of acknowledging the device. A handler that doesn't release its line is entered again as soon as it returns.
- **NMI is an edge:** asserting it requests one NMI.
- **When:** a pending interrupt is taken between instructions, before the next one, if it isn't masked: NMI always (it is not disarmed until S is loaded, unlike the real chip), FIRQ if F is clear, IRQ if I is clear. NMI comes first, then FIRQ, then IRQ.

- **Stacking**, as on the MC6809:

Interrupt	Stacked	Masks set	Vector
IRQ	all registers, E = 1	I	`$FFF8`
FIRQ	PC and CC, E = 0	I, F	`$FFF6`
NMI	all registers, E = 1	I, F	`$FFFC`

- **The timer** asserts the lines selected by +10 (IRQ unless changed) every N instructions, counting the handlers' instructions too. It is a simple way to interrupt a program at many different points.
- **`CWAI #mask`** ANDs CC with the mask, stacks all registers with E set, and waits; the interrupt then doesn't stack them again. **`SYNC`** waits until an interrupt line is asserted: if it is masked, execution just continues after `SYNC`. Waiting counts against `-n` like instructions, so a wait with no interrupt to come ends at the instruction limit.

With `-t`, a line such as `---- IRQ` is printed when an interrupt is taken.

6.3 Instructions that stop the simulator

Unless `--swi` is given, software interrupts stop the simulator (most test programs use none, and one there is usually a bug), along with anything that isn't a valid MC6809 instruction:

Instruction	Message
`SWI`, `SWI2`, `SWI3` (without `--swi`)	`SWI`, `SWI2`, `SWI3`
undocumented opcodes (e.g. `$01`, `$02`, `$10 $00`)	`illegal opcode`
`$87`, `$8F`, `$C7`, `$CD`, `$CF`, `$10 $8F`, `$10 $CF`	`illegal opcode (store immediate)`
reserved indexed postbytes (`[,R+]`, `[,-R]`, modes 7, A and E, or `$8F`/`$AF`/… with no indirection)	`illegal indexed mode` / `illegal indexed postbyte`

In each case the simulator prints the message, the address of the offending instruction and all registers to standard error, then exits with status 125:

```
sim6809: illegal opcode at PC=1001
  A=00 B=00 X=0000 Y=0000 U=0000 S=0000 DP=00 CC=-F-I----
```

The undocumented MC6809 opcodes (for example `$01` as a NEG alias, or `$3E` RESET) are treated as illegal rather than emulated. Code that relies on them is almost always a bug.

7 OS-9 mode

With `--os9`, `sim6809` runs a 6809 **OS-9 program module** (for example one made by `plmc --os9` and `lwasm -f os9`) as if OS-9 had started it. Nothing else about the simulator changes: the trace, the instruction limit and the I/O block all still work.

```
build/sim6809 --os9 prog
build/sim6809 --os9 --load 0x3123 --data 0x9E00 prog < input.txt
```

Loading. The whole file is read and checked. Loading fails with exit status 2 if any check fails:

- the `$87CD` sync bytes;
- the module size in the header, which must not exceed the file;
- the header parity (bytes 0–8 must XOR to `$FF`);
- the 24-bit module CRC;
- the type/language byte, which must be `$11` (a 6809 object-code program module).

The module is copied to `--load` unchanged, so a module that isn't position-independent shows up as soon as it's loaded somewhere other than where it was assembled for.

Process setup follows `F$Fork`:

Item	Value
data area	`--data`, with the size from the module header plus the parameter string, rounded up to whole pages
parameters	the `--param` string followed by a CR, at the top of the data area
U	start of the data area
X, S	start of the parameters
Y	end of the data area
D	length of the parameters, including the CR
DP	the data area's page (`--data` / 256)
CC	0
PC	the module's execution entry

System calls. `SWI2` followed by a function byte (`OS9 F$xxx` in lwasm) calls one of these:

Call	Code	Emulation
`F$Exit`	`$06`	Stop; B becomes the simulator's exit status.
`F$Mem`	`$07`	Make the data area D bytes, rounded up to whole pages (0 asks its size), growing or shrinking it at its end; returns D = its size and Y = its end. `E$MemFul` (`$CF`) if it would reach the module or the I/O block. Memory protection follows it.
`I$Write`	`$8A`	Write Y bytes from X to path A: 1 = stdout, 2 = stderr, or a file. Returns Y = bytes written.
`I$WritLn`	`$8C`	Like `I$Write`, but stops after the first CR.
`I$Read`	`$89`	Read up to Y bytes from path A (0 = stdin, or a file or directory) into X. Returns Y = bytes read.
`I$ReadLn`	`$8B`	Like `I$Read`, but stops after the first CR.

Call	Code	Emulation
I$Open	$84	Open the host file named at X in mode A (1 read, 2 write, 3 update). Returns the path in A, and X past the name. With DIR. ($81), a directory is opened, to be read as RBF's: 32-byte entries, .. and . first and then the names in order, each name with bit 7 set in its last character and followed by a 3-byte LSN (made up). A directory opened without DIR. gives E$FNA ($D6), and a file opened with it E$BMode.
I$Create	$83	Like I$Open, but makes a new, empty file (B, the attributes, is ignored); an existing one gives E$CEF.
I$Close	$8F	Close path A.
I$Seek	$88	Move path A's position to X:U (ignored on the terminal).
I$GetStt	$8D	Status B of path A: SS.Opt ($00) puts 32 option bytes at X, the first the device type, 0 (SCF) for the terminal and 1 (RBF) for a file; SS.Size ($02) and SS.Pos ($05) return a file's size and position in X:U; SS.EOF ($06) gives E$EOF at a file's end.
I$SetStt	$8E	Status B of path A: SS.Size ($02) makes a file X:U bytes long; SS.Opt is accepted and changes nothing.
I$Delete	$87	Delete the host file (or empty directory) named at X.
I$MakDir	$85	Make the host directory named at X (B, the attributes, is ignored); an existing one gives E$CEF.
I$ChgDir	$86	With READ. or WRITE. in A, the directory named at X becomes the current (data) directory, which host names are relative to; EXEC. alone is accepted and changes nothing.

- **Paths:** paths 0, 1 and 2 are the terminal: stdin, stdout and stderr (stdout is flushed before a read and before a write to stderr, so that output sent to one file keeps its order). I$Open and I$Create give paths 3 to 15. A path name is the letters, digits, ., _, $ and / at X, relative to the current directory.
- **Newlines:** OS-9 ends lines with CR. On the terminal, output CR is written as a newline, and an input newline is delivered as CR, the way a terminal behaves under OS-9. Files are read and written as they are.
- **Errors:** a call that fails returns with carry set and the error code in B. A bad path number gives E$BPNum ($C9), reading at end of input or of a file E$EOF ($D3), a file that can't be opened or deleted

E$PNNF ($D8), a bad mode E$BMode ($CB), a full path table E$PthFul ($C8), a status code not listed above E$UnkSvc ($D0), and a failed write E$Write ($F5).

- **Other calls:** any other function code stops the simulator with `unsupported OS-9 system call $xx` and exit status 125.

Memory protection. In OS-9 mode the simulator enforces what a module may touch. Each of these stops it with exit status 125 and a register dump:

Access	Allowed	Message otherwise
data read	the module and the data area	`read outside the module and data area`
data write	the data area	`write outside the data area`
instruction fetch	the module	`PC outside the module`

These catch the two classic position-independence bugs: an absolute JSR/JMP to a label, and an absolute variable address. Direct-page references (`LDA <$10`) are legitimate under OS-9, because DP points at the data area.

Not emulated. OS-9 mode does not provide other system calls, other modules or subroutine libraries, signals, multitasking, file attributes or ownership, the execution directory, or a real OS-9 file system. For full-system testing, the plm09 test suite also boots NitrOS-9 on an emulated CoCo 3 under XRoar (see the plmc manual).

8 FLEX mode

With `--flex`, `sim6809` runs a **FLEX binary file**, such as a `.CMD` command made by plmc's `srec2flex`, as if FLEX 9 had loaded it.

```
build/sim6809 --flex [--param STR] HELLO.CMD < input.txt
```

Loading. The file is a sequence of $02 records (load address, count, data) and a $16 transfer address record, with $00 padding allowed between records, as the FLEX Advanced Programmer's Guide describes. Loading fails with exit status 2 if the file holds anything else, if a record runs past its end, or if there is no transfer address. The program then starts at the transfer address with S = $C07F (as FLEX starts commands), CC = 0 and the other registers 0; the end of user memory ($CC2B) reads $BFFF. The line buffer ($C080) holds the command line as FLEX leaves it: the file's name in upper case, without its directory and extension, then a blank and the `--param` string if there is one, and a CR; the line buffer pointer ($CC14) points past the name and the blank after it, at the parameters (or at the CR if there are none), where real FLEX leaves it.

DOS entry points. A call (or jump) to an entry point of FLEX's jump table is emulated, and returns as an RTS would. Y and U are preserved, as FLEX promises:

Entry	Address	Emulation
WARMS, RENTER	$CD03, $CD06	Back to FLEX: stop, with B as the exit status (a sim6809 convention; FLEX itself has none).
INCH, INCH2, GETCHR	$CD09, $CD0C, $CD15	A = a character from stdin; a newline is delivered as CR, and end of input as 0.

Entry	Address	Emulation
OUTCH, OUTCH2, PUTCHR	$CD0F, $CD12, $CD18	Write A to stdout.
PSTRNG	$CD1E	A new line, then the string at X up to EOT ($04).
CLASS	$CD21	Carry clear if A is a letter or digit; otherwise carry set and A stored in Last Terminator ($CC11).
PCRLF	$CD24	A new line.
RSTRIO	$CD2A	Nothing.
ADDBX	$CD36	X = X + B.
OUTDEC	$CD39	Print the 16-bit number at X in decimal, right-aligned in 5 columns if B is non-zero.
OUTHEX, OUTADR	$CD3C, $CD45	Print the byte (2 digits) or word (4 digits) at X in hex.

- **Newlines:** FLEX terminals end lines with CR LF. Output CR is written as a newline and output LF is dropped, so PCRLF, or CR followed by LF, give one newline.
- **Other entry points** (COLDS, INBUFF, NXTCH, GETFIL, LOAD, DOCMND) stop the simulator with `unsupported FLEX call NAME ($CDxx)` and exit status 125.

Files. A call to the File Management System (FMS, $D406, with X pointing at a File Control Block) works on host files, in the current directory, named by the FCB's name and extension (NAME.EXT; the drive is ignored). A host file holds the data bytes of FLEX's sectors, as `flexdisk.py get` reads them off a FLEX disk: text compressed as FLEX stores it, and zero-padded to whole sectors of 252 bytes. As FLEX's FMS does, it returns with Z set if all went well, or with Z clear and the error number in the FCB's byte 1, keeping B, X, Y and U.

Function	Emulation
0, read or write the next byte	A is the byte. With the FCB's space compression flag (byte 59) 0, a text file: a run of 2 to 127 spaces is written as $09 and the count, and read back as spaces, and NULs are skipped when reading; with $FF, a binary file, bytes are written and read as they are. Error 8 at the end of the file.
1, open for read	Error 4 if there is no such file.
2, open for write	A new file: error 3 if it exists.
4, close	A written file is padded with zeros to whole sectors; one closed with nothing written is removed, as FLEX removes it.
5, rewind	A file open for read; the compression flag becomes 0 (text), as FLEX makes it.
12, delete	Error 4 if there is no such file; the FCB's name's first byte becomes $FF, as FLEX marks it (so it must be set again before the FCB is used).
13, rename	To the name in bytes 53 to 63: error 4 if there is no file, 3 if the new name exists.

FMSCLS ($D403) closes every open file, and FMSINI ($D400) does nothing. Other functions stop the simulator with `unsupported FMS function N` and exit status 125. Errors 2 (already open), 13 (the FCB isn't open), 18 (a read on a file open for write, or the reverse) and 21 (no name) are given too.

Memory protection. FLEX occupies `$C700`–`$DFFF` (below it are FLEX's stack, its line buffer, and the utility command area `$C100`–`$C6FF`, which belongs to the running command). A write there stops the simulator with `write to the operating system's memory`, and code run there other than at an entry point with `code run in FLEX's memory, not at a DOS entry point`, both with exit status 125. The I/O block still works, and nothing else changes.

9 Board devices

These options let the simulator stand in for small 6809 boards, so that a board's own monitor or BASIC ROM, or programs written for it, can run unchanged. They can be combined with each other and with the I/O block (move it with `--io` if it would overlap the board's memory map). With `--rom` the ROM area ignores writes, as a real ROM does (BASICs find the end of RAM by writing to it), and `--reset` starts the ROM from its reset vector.

- **`--swi:`** `SWI`, `SWI2` and `SWI3` work as on the MC6809: all registers are stacked with E set, `SWI` also sets I and F, and execution continues at the address in `$FFFA`, `$FFF4` or `$FFF2`. Monitors use `SWI` for breakpoints and for programs to return to them. (In OS-9 mode `SWI2` stays a system call.)
- **`--acia ADDR:`** an MC6850 ACIA, the serial port of many boards.
  - *ADDR* reads as the status register: TDRE (bit 1, ready to send) is always set, and RDRF (bit 0, a character is waiting) when standard input has one, but only once the program has read the status 32 times since its last data access. That is how a person types: after a prompt, not while the program is still printing. It matters for monitors that check for a key between output characters (to pause or abort), which would otherwise swallow the input. Writes to *ADDR* (control) are ignored.
  - *ADDR*+1 is the data register: reads take the next character from standard input, and writes go to standard output.
  - At the end of standard input the data register reads 0 (and RDRF is set), as the I/O block's input port does, so a program sees the end instead of waiting forever. With `--stop-at-end`, a program that waits for input then (polling the status) ends the run with exit status 0 instead: the end of a monitor or BASIC session.
- **`--ft245 ADDR:`** an FTDI FT245RL USB FIFO, as on the Simon6809 board.
  - Reads of *ADDR* take the next byte of standard input (0 if there is none), and writes go to standard output.
  - The chip's "data available" line drives **FIRQ**: it is asserted while standard input has a byte waiting, and taken (or ends a `SYNC`) as an asserted line does.
  - When the program waits for an interrupt (`SYNC` or `CWAI`) after the end of standard input, with no other source that could interrupt, the run ends with exit status 0: the natural end of an interactive session.

For example, this runs the Simon6809 board's monitor ROM, which starts at `$E000` and uses `SWI` for breakpoints, with its console on the terminal (the I/O block moved out of the ROM's way):

```
build/sim6809 --swi --io 0xD800 --ft245 0xD000 --rom 0xE000 --entry 0xE000 simon09.rom.s19
```

and this Grant Searle's 6809 SBC with his BASIC ROM, an Intel HEX file:

```
build/sim6809 --acia 0xA000 --stop-at-end --io 0x9000 --rom 0xC000 --reset ExBasROM.hex
```

10 Exit status

Status	Meaning
0–255	The program wrote this value to the exit port (+2).
124	The instruction limit (`-n`) was reached. A message on stderr gives the PC.
125	Illegal opcode or postbyte; `SWI`/`SWI2`/`SWI3` without `--swi` (in OS-9 mode, `SWI2` is a system call); an unsupported OS-9 call or FLEX entry point; or an OS-9 or FLEX memory protection violation. A register dump goes to stderr.
2	Usage error, unreadable file, bad S-record, OS-9 module or FLEX binary, or no start address.

A program can itself exit with 124, 125 or 2, so a script that needs to tell these cases apart should also check standard error. (124 is also the status the `timeout` command uses when it stops a command.)

A program that never writes the exit port runs until the instruction limit:

```
$ sim6809 -n 5 hello.s19
Hsim6809: instruction limit (5) reached at PC=100E
$ echo $?
124
```

11 Tracing

With `-t`, the simulator prints one line to standard error **before** each instruction executes:

```
1000 10CE8000   LDS   #$8000         A=00 B=00 X=0000 Y=0000 U=0000 S=0000 DP=00 CC=-F-I----
1004 8E101B     LDX   #$101B         A=00 B=00 X=0000 Y=0000 U=0000 S=8000 DP=00 CC=-F-IN---
1007 A680       LDA   ,X+            A=00 B=00 X=101B Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
1009 2705       BEQ   $1010          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
100B B7FF00     STA   $FF00          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
100E 20F7       BRA   $1007          A=48 B=00 X=101C Y=0000 U=0000 S=8000 DP=00 CC=-F-I----
```

The columns are:

1. **Address** of the instruction.
2. **Instruction bytes** in hex: opcode, any prefix, postbyte and operands, up to 5 bytes.
3. **Disassembly** in lwasm-compatible syntax.
   - Direct-page operands are shown as `<$nn`.
   - Branch targets and PC-relative operands are shown as the absolute address they refer to.
   - Undecodable bytes are shown as `???`.
4. **Registers** as they are *before* the instruction runs.
   - CC is spelled out as `EFHINZVC`, with `-` for a clear flag.

Because the trace goes to standard error and program output to standard output, you can separate them:

```
sim6809 -t prog.s19 > output.txt 2> trace.txt
```

Programs often run millions of instructions, so a trace can get large. A lower `-n` limit, `head`, or `grep` on an address range keeps it manageable. For example, to see what a routine at `$1234` was called with:

```
sim6809 -t prog.s19 2>&1 >/dev/null | grep '^1234 '
```

12 Using sim6809 with plmc

plmc's simulator hooks (`runtime/sim/plm_hooks.asm`) implement the PL/M `EXTERNAL` procedures `PUTCH`, `GETCH`, `PUTDEC` and `EXIT` with the I/O block above, and put the stack top at `$FF00`. A PL/M program can also reach the I/O block directly as an array AT `$FF00` (`DECLARE IO(16) BYTE AT (0FF00H);`), so `IO(0) = C` prints a character, `IO(1)` reads one, and the interrupt and timer registers are `IO(6)` to `IO(10)`. The whole cycle is:

```
build/plmc -o prog.asm prog.plm
lwasm -9 -f srec -I runtime -I runtime/sim -o prog.s19 prog.asm
build/sim6809 prog.s19 < input.txt
```

`make run F=prog.plm` does the same in one step. When a program misbehaves, the generated `.asm` has each PL/M statement as a comment (`; 12: X = X + 1;`), and the trace addresses can be matched against lwasm's listing (`--list=prog.lst`).

13 Verification

The simulator is checked by `make sim-test`, which runs:

- **Hand-checked tests.**
  - *ALU:* the 8- and 16-bit arithmetic and logic instructions, checked result and flags against tables worked out by hand from the datasheet, with boundary values such as `$7F`, `$80`, `$FF` and `$00`.
  - *Branches:* all 16 short and 16 long conditional branches, each tried with all 16 combinations of N, Z, V and C.
  - *Indexed addressing:* every indexed form, including indirect and PC-relative, plus direct addressing with a non-zero DP.
  - *Stack and transfers:* `PSHS`/`PULS`/`PSHU`/`PULU` ordering, `JSR`, `BSR`, `LBSR`, `RTS`, `PULS PC`, `RTI` with E clear and set, and `TFR`/`EXG`.
  - *Edge cases:* an illegal opcode and the instruction limit, which must give exit statuses 125 and 124.
  - *Interrupts:* IRQ, FIRQ and NMI with their masks, stacking, vectors and registers seen inside the handler; priority; the timer (with the exact number of loop passes before its third interrupt); timer-driven FIRQ and NMI; `CWAI` and `SYNC`.
- **OS-9 mode:**
  - a system-call test loads a module at the unaligned address `$3123` with data at `$9E00`, then checks every `F$Fork` register, each emulated call, and the error codes;
  - a file test creates, writes, reads, seeks in, truncates and deletes a host file, and checks each status code and error;
  - a directory test checks `F$Mem` (asking, growing, writing the new memory, too much, shrinking), and makes a directory, creates a file in it, changes into it, lists it, changes back and deletes both;
  - two more tests check that an absolute write and an absolute `JMP` are stopped by memory protection.

- **Board devices:** a test checks `SWI`, `SWI2` and `SWI3` with `--swi` (vectors, masks, the 12 bytes stacked with E set), the ACIA's status and data registers at the end of input (including the 32 status reads before RDRF), and the FT245's reads and writes, ending in `SYNC` with no input to come.
- **FLEX mode:** a test calls every emulated entry point and checks the start-up state, that Y and U survive, and `CLASS`'s carry and Last Terminator; three more check that a write to FLEX's memory, a jump into it and an unemulated entry point stop the program; one checks the command line in the line buffer; and one checks the File Management System: a text file written with a run of spaces, read back as text and as binary (its compressed, padded bytes), errors, renaming and deleting.
- **Instruction-length check against lwasm.** A generated program contains every opcode in every addressing mode, about 11,000 instructions, laid out so execution falls straight through. The program is run with `-t`. Every traced address and its instruction bytes must match the address and code columns of lwasm's listing. This checks the instruction decoder in the CPU and the one in the disassembler independently, against the assembler.
- **Mutation checks.** During development, bugs were introduced on purpose (wrong flags, a 16-bit offset decoded as 8-bit) to confirm that the tests catch them.

14 Limitations

- **OS-9 mode is minimal.** Only the fifteen system calls above are emulated (see OS-9 mode).
- **FLEX mode is minimal.** Only the terminal and number routines are emulated; there is no command line, disk or file system (see FLEX mode).
- **Simple interrupts.** The only interrupt sources are the I/O block's lines and timer, and the FT245's FIRQ; NMI is never disarmed; `SWI`, `SWI2` and `SWI3` stop the simulator unless `--swi` is given.
- **Simple devices.** The ACIA and FT245 have no baud rates, timing, flow control or error flags.
- **No timing.** Instructions take no cycles, so timing loops run instantly and cycle counts aren't reported.
- **No 6309.** The Hitachi 6309's native mode and extra instructions aren't supported; they stop the simulator as illegal opcodes.
- **No real hardware.** There is no ROM, no video or other machine-specific hardware, no bank switching, and no memory beyond 64 KB. For machine-accurate emulation of a Color Computer or Dragon, use XRoar or MAME.
- **One input format.** Only S1/S9 S-records are loaded. Convert other formats with lwasm or a tool such as `srec_cat`.
- **Memory is not saved.** The memory image can't be inspected or saved after the run; print values through the I/O block instead.

15 Source code

File	Contents
`sim/cpu6809.c`, `cpu6809.h`	CPU core: registers, flags, addressing modes, instruction execution, I/O block
`sim/disasm.c`	table-driven disassembler used by the trace
`sim/srec.c`, `srec.h`	S-record loader
`sim/os9.c`, `os9.h`	OS-9 mode: module loading, process setup, system calls
`sim/main.c`	command-line handling, run loop, trace and diagnostics
`sim/tests/`	self-test programs and expected output; `harness.inc` provides the check macros, `gen_len_ops.py` generates the length test

The core can be embedded in another program:

- `cpu_init(&cpu, io_base, fill)` resets the CPU and memory.
- Set `cpu.pc` and fill `cpu.mem`.
- Call `cpu_step(&cpu)` until it returns a non-zero `STOP_*` reason.
- `disasm(mem, pc, buf, len)` formats one instruction and returns its length.